

Week 8: Hands-On Project Week

Scaler School of Technology | Deep Learning I: Neural Networks | Week 8 | 3 Sessions



What Is This Week About?

Week 8 is your chance to **build, experiment, and connect the dots**. Over three hands-on sessions you will implement six mini-projects — each one consolidating a different arc of the course. There are no new theory topics this week. Take everything you have learned from Weeks 1–7 and make it real: running code, comparing models, visualising internals, and developing intuition for *when* and *why* each architecture works. Session 3 goes one step further — you will train a real language model from scratch and fine-tune a pretrained GPT-2 using LoRA.

Session Structure

#	Project	Topics Consolidated	Session	GPU?
1	ANN: Regression + Classification	Weeks 1–3: MLP, backprop, loss, regularisation	Session 1	No
2	Neural Style Transfer (VGG19)	Weeks 4–5: CNNs, transfer learning, feature maps	Session 1	Yes
3	Shakespeare Text Generator (LSTM)	Weeks 5–6: RNNs, LSTMs, sequence modelling	Session 2	Yes
4	English→French Translation (Seq2Seq)	Week 6: Encoder-decoder, attention mechanism	Session 2	Yes
5	Mini-Transformer from Scratch	Week 7: Self-attention, positional encoding, full arch	Session 3	Yes
6	Train a Small Language Model (nanoGPT + LoRA)	Weeks 6–7: LM pretraining, fine-tuning, LoRA	Session 3	Yes

Learning Objectives

- Apply MLPs to real tabular datasets and compare against classical ML baselines ✓
- Exploit pre-trained CNN features for artistic style transfer ✓
- Build a character-level language model with LSTMs and control generation with temperature ✓
- ✓ Implement Bahdanau attention and visualise alignment heatmaps ✱
- ✓ Assemble a complete Transformer encoder-decoder from scratch
- ✓ Train a GPT-style language model from scratch on WikiText-103 (nanoGPT, ~10M params)
- ✓ Fine-tune a 117M-parameter pretrained model by updating only 0.6% of its weights (LoRA)
- ✱ Understand *why* each architecture is the right tool for each problem

Prerequisites You Should Have

- **PyTorch basics:** tensors, `nn.Module`, training loops
- **MLPs:** forward pass, loss, SGD/Adam, backprop
- **CNNs:** convolutions, pooling, feature maps, pre-trained models
- **RNNs/LSTMs:** hidden state, gates, sequence-to-sequence
- **Attention:** context vector, alignment scores, Bahdanau
- **Transformers:** scaled dot-product attention, multi-head, positional encoding
- **Language modelling:** autoregressive generation, cross-entropy loss, perplexity

How to Prepare (Optional but Helpful)

- ✓ Ensure your Google Colab account has GPU access (*Runtime* → *Change runtime type* → *T4 GPU*)
- ✓ Refresh the PyTorch training loop pattern: `define model` → `forward` → `loss` → `backward()` → `step()`
- ✓ Optionally skim Karpathy's char-rnn blog post before Session 2 (Project 3)
- ✓ Re-read your Session 2 Week 7 notes on multi-head attention before Session 2
- ✓ Skim Andrej Karpathy's "Let's build GPT" YouTube lecture before Session 3
- ✓ Install `peft` and `transformers` in Colab: `!pip install peft transformers`

Duration: ~1.5 hrs per project · 3 sessions

Instructor: Priyansh Saxena

Tech Stack: Python 3.10+, PyTorch, scikit-learn, Google Colab

Session 1 — Projects 1 & 2

Scaler School of Technology | Deep Learning I: Neural Networks | Week 8



Project 1 — ANN: Regression + Classification vs Classical ML

What you will build: Two PyTorch MLPs — one that predicts California house prices (regression) and one that diagnoses heart disease (classification). You will benchmark both against classical ML baselines (Linear Regression, Random Forest, Logistic Regression, SVM) and develop an intuition for *when* deep learning actually helps.

A Part A: Regression — California Housing

Load the California Housing dataset (`sklearn.datasets.fetch_california_housing`). Fit a Linear Regression and Random Forest baseline, then train a 2-hidden-layer MLP (64–32 units, ReLU, Adam, 100 epochs). Compare MSE and R^2 across all three models with a bar chart. *Spoiler: the MLP won't blow the baselines out of the water — and that's the lesson.*

B Part B: Classification — Heart Disease Prediction

Load the UCI Heart Disease dataset (~300 samples, 13 features). Fit Logistic Regression and an SVM baseline. Train an MLP classifier (64–32, ReLU, Dropout 0.3, sigmoid output, BCELoss, 150 epochs). Compare Accuracy, F1, and AUC-ROC. Plot the ROC curve and confusion matrix. *Small structured data is classical ML's home turf.*

C Reflection: When Does DL Win?

After both comparisons, discuss: diminishing returns on small tabular data, when scale and non-linearity tip the balance toward deep learning, and why you should always establish simple baselines first. These are professional habits.

Key Concepts Practised

- PyTorch training loop from scratch
- `StandardScaler` + tensor conversion pipeline
- MSE loss, BCELoss, Adam optimiser
- Train / val / test split and evaluation metrics
- ROC-AUC and confusion matrix
- Model complexity vs. dataset size trade-off

Exercises (if you finish early)

- Add L2 weight decay (`weight_decay` in Adam) to the regression MLP — does it reduce overfitting?
- Try a 4-layer deep MLP on the classification task — does more depth help on 300 samples?
- Add a learning rate scheduler (`StepLR` or `ReduceLROnPlateau`) and compare convergence curves

Duration: ~90 minutes

GPU: Not required

Dataset: `sklearn.datasets` (no download)

Notebook:

`Session_1_ANN_Regression_Classification.ipynb`

Session 1 — Projects 1 & 2

Scaler School of Technology | Deep Learning I: Neural Networks | Week 8



Project 2 — Neural Style Transfer with VGG19

What you will build: A style transfer pipeline that takes two images — a *content* image (a photo) and a *style* image (a famous painting) — and produces a third image that preserves the content's structure while adopting the painting's texture. You won't train a new network; you'll *optimise the pixels of the output image* using a frozen pre-trained VGG19.

1 Content and Style Representations

Load VGG19 with pre-trained ImageNet weights. Freeze all parameters. Extract feature maps at selected convolutional layers: conv4_2 captures content (high-level structure); conv1_1, conv2_1, conv3_1, conv4_1, conv5_1 capture style (textures). Visualise what each layer “sees” on a sample image.

2 Loss Functions: Content, Style, Total Variation

Content loss: MSE between content image features and generated image features at conv4_2. **Style loss:** MSE between *Gram matrices* (feature correlations) of the style and generated images across style layers. **TV loss:** smoothness regulariser to reduce pixel noise. Total loss = $\alpha L_{\text{content}} + \beta L_{\text{style}} + \gamma L_{\text{TV}}$.

3 Optimising the Image (not the Weights)

Initialise the generated image as a copy of the content image. Run LBFGS on the *pixels* of the generated image for 300 iterations, computing the full loss at each step and backpropagating through the frozen VGG19. Display intermediate results every 50 steps. Final output: content — style — result side by side.

Key Concepts Practised

- Extracting intermediate feature maps from a pre-trained CNN
- Gram matrix as a texture descriptor
- Gradient flow through a frozen network to inputs
- LBFGS optimiser for image optimisation
- Balancing multiple loss terms with hyperparameters
- Hierarchical representations: texture → shapes → objects

Experiments to Try

- Vary α/β : increase β for a more painterly result; increase α to preserve photo realism
- Swap style images: try Van Gogh's *Starry Night*, Hokusai's *The Great Wave*, a Picasso cubist work
- Use only shallow style layers (conv1_1, conv2_1) — how does this change the texture scale?
- Initialise with random noise instead of the content image — what changes about convergence?

Duration: ~90 minutes **GPU:** Required (Colab T4) **Model:** VGG19 pre-trained on ImageNet (~548 MB download)

Notebook: Session_2_Neural_Style_Transfer.ipynb

Session 2 — Projects 3 & 4

Scaler School of Technology | Deep Learning I: Neural Networks | Week 8



Project 3 — Shakespeare Text Generator (Character-Level LSTM)

What you will build: A character-level language model trained on Andrej Karpathy's *Tiny Shakespeare* dataset (~1 MB). The model reads sequences of characters and predicts the next one. After training, you feed it a prompt like "ROMEO:" and it generates new text — letter by letter — in a vaguely Shakespearean style. You'll also compare LSTM against vanilla RNN to see vanishing gradients in action.

1 Data Pipeline & Vocabulary

Download Tiny Shakespeare. Build a `char2idx / idx2char` mapping over all unique characters (~65). Encode the full text as an integer sequence. Create sliding-window training sequences of length 100: input characters `[0:100]`, target `[1:101]`. Wrap in a PyTorch Dataset and DataLoader (batch size 64).

2 CharLSTM Model & Training

Architecture: `Embedding(65, 128) → LSTM(128, 256, 2 layers, dropout 0.2) → Linear(256, 65)`. Loss: `CrossEntropyLoss`, Adam `lr=0.002`, gradient clipping at 5. Train for 20 epochs, printing sample generated text every 5 epochs to see the model improve from gibberish to quasi-English to Shakespeare-like structure.

3 Temperature Sampling & LSTM vs RNN

Temperature: divide logits by T before softmax. $T=0.2$: conservative and repetitive. $T=0.8$: balanced and creative. $T=1.5$: chaotic and random. Compare outputs side-by-side. **LSTM vs RNN:** swap the LSTM for a vanilla RNN with the same hidden size. Train and compare loss curves and generated text quality — vanishing gradients become concrete.

Key Concepts Practised

- Character-level tokenisation (no tokeniser needed)
- Autoregressive generation: feeding outputs back as inputs
- LSTM gating (forget / input / output gates)
- Gradient clipping to prevent exploding gradients
- Temperature as exploration–exploitation dial
- Concrete evidence of vanishing gradients: RNN vs. LSTM

Exercises (if you finish early)

- Replace LSTM with a GRU — is there a speed or quality difference?
- Train on a different corpus (Python source code, song lyrics, news headlines) — observe how quickly the model picks up the style
- Implement top- k sampling as an alternative to temperature (sample only from the k highest-probability characters)

Duration: ~90 minutes **GPU:** Required (Colab T4) **Dataset:** Tiny Shakespeare (downloaded in-notebook, ~1 MB)

Notebook: `Session_3_Shakespeare_Text_Generator.ipynb`

Session 2 — Projects 3 & 4

Scaler School of Technology | Deep Learning I: Neural Networks | Week 8



Project 4 — English→French Translation (Seq2Seq + Bahdanau Attention)

What you will build: A sequence-to-sequence neural machine translation model with Bahdanau (additive) attention. The encoder reads an English sentence and produces a sequence of hidden states. The decoder generates French words one at a time, using an attention mechanism to decide which encoder states to focus on at each step. You will produce interpretable attention heatmaps that show word-to-word alignment.

1 Data & Vocabulary

Load Tatoeba English–French sentence pairs (available at manythings.org/ or [anki](https://ankiweb.com/)). Filter to short sentences (≤ 10 words) and simple patterns — around 10 000 pairs after filtering. Build separate `word2idx` / `idx2word` vocabularies for English and French, both including `<PAD>`, `<SOS>`, and `<EOS>` tokens. Pad and batch into a `Dataloader` (batch size 64).

2 Encoder, Attention Module, & Decoder

Encoder: $\text{Embedding}(\text{vocab}, 256) \rightarrow \text{Bidirectional GRU}(256, 256) \rightarrow \text{outputs all hidden states}$. **Attention:** $\text{score} = v^\top \tanh(W_1 h_{\text{dec}} + W_2 h_{\text{enc}}) \rightarrow \text{softmax} \rightarrow \text{context} = \sum \alpha_i h_{\text{enc}, i}$. **Decoder:** $\text{Embedding} \rightarrow \text{GRU with [emb; context] input} \rightarrow \text{Linear to vocab}$. Teacher forcing ratio 0.5 during training.

3 Attention Heatmaps & Failure Cases

After 25 training epochs, translate held-out English sentences and compute the BLEU score. Plot attention weight matrices as heatmaps (English words on x -axis, French output words on y -axis). The model should show clean diagonal alignment for simple sentences. Study failure cases: long sentences, rare words, French–English word order differences.

Key Concepts Practised

- Encoder-decoder architecture and the fixed-vector bottleneck
- Bahdanau additive attention: score, alignment, context vector
- Teacher forcing and curriculum learning
- BLEU score as a translation quality metric
- Interpretable attention: the model tells you what it's looking at
- Limitations that motivate the Transformer (sequential, slow)

Experiments to Try

- Compare against a plain seq2seq (no attention) on the same data — quantify the quality gap
- Implement Luong (multiplicative) attention: $\text{score} = h_{\text{dec}}^\top W h_{\text{enc}}$ — how does it compare?
- Implement beam search (beam width 3) instead of greedy decoding and check BLEU improvement

Duration: ~90 minutes **GPU:** Required (Colab T4) **Dataset:** Tatoeba EN–FR, filtered to ~10 000 pairs **Notebook:** `Session_4_Seq2Seq_Attention_Translation.ipynb`

Session 3 — Projects 5 & 6

Scaler School of Technology | Deep Learning I: Neural Networks | Week 8



Project 5 — Mini-Transformer from Scratch

What you will build: A complete Transformer encoder-decoder, implementing *every component yourself* — no HuggingFace, no pre-built attention layers. Task: convert digit strings to English words ("123" → "one two three"). Tiny vocab, fast training — the architecture is the focus. By the end you'll understand the internals of GPT and BERT at the component level.

1 Building Blocks: Attention & Encoding

Positional encoding: $PE(pos, 2i) = \sin(pos/10000^{2i/d})$. **Scaled dot-product:** $Attn(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$. **Multi-head:** split d_{model} into n_{heads} subspaces, run attention in parallel, concatenate, project. **FFN sublayer:** Linear → ReLU → Linear. Each sublayer wrapped in Add & LayerNorm.

2 Encoder, Decoder & Causal Masking

Encoder: self-attention → Add&Norm → FFN → Add&Norm. Stack 2 layers. **Decoder:** *masked* self-attention → Add&Norm → cross-attention (attends to encoder) → Add&Norm → FFN → Add&Norm. Stack 2 layers. **Causal mask:** upper-triangular boolean matrix prevents attending to future tokens.

3 Training, Evaluation & Attention Visualisation

Train 50 epochs (Adam + linear warmup). Evaluate exact-match accuracy on numbers 801–999. Visualise: encoder self-attention (digit–digit), decoder self-attention (word–word), cross-attention (which digits each output word focuses on). Compare heads — each learns a different pattern.

Key Concepts Practised

- Scaled dot-product attention (the core operation)
- Multi-head: multiple “perspectives” simultaneously
- Positional encoding: order without recurrence
- Residual connections + layer normalisation
- Causal masking: no peeking at future tokens
- LR warmup for stable Transformer training

Experiments to Try

- Remove positional encoding — what breaks?
- 1 head vs. 4 vs. 8 — quality difference?
- 4 encoder/decoder layers — better or overfit?
- Time step vs. Project 4 RNN (parallelism!)
- ★ Learned vs. sinusoidal positional embeddings

Course Arc

Perceptron → MLP + Backprop → CNN → RNN + LSTM → Seq2Seq + Attention → Transformer → **Week 8: You build all of it.**

Duration: ~90 minutes **GPU:** Recommended (Colab T4) **Dataset:** Self-generated numbers 1–999, no download
Notebook: Session_5_Transformer_From_Scratch.ipynb

Session 3 — Projects 5 & 6

Scaler School of Technology | Deep Learning I: Neural Networks | Week 8



Project 6 — Training a Small Language Model: nanoGPT + LoRA

What you will build: Two language models in one session — at opposite ends of the training spectrum. In **Part 1** you train a GPT-style decoder-only transformer from random weights on **WikiText-103** (~10M params, ~8 minutes). In **Part 2** you load a pre-trained GPT-2 (117M params) and fine-tune it on Python code by updating only **0.6% of its weights** using LoRA. Both generate text; the contrast in quality and approach is the lesson.

1 Part 1 — nanoGPT: Pretraining from Scratch

Build a decoder-only transformer (same building blocks as Session 5, but decoder-only on real text). Architecture: 6 layers, 8 heads, $d_{\text{model}} = 512$, context length 256, ASCII character-level tokeniser. Train on WikiText-103 (~30MB) with cosine LR decay. Generate text every 500 steps to watch the model go from gibberish to coherent prose. *Larger model + more diverse data than Session 3 — directly comparable architecture.*

2 Part 2 — GPT-2 + LoRA: Efficient Fine-Tuning

Load GPT-2 small (117M params, trained on 40B tokens). Freeze all weights. Inject LoRA adapters ($r = 8$) into the attention layers: $W_{\text{new}} = W_{\text{frozen}} + BA$ where $A \in \mathbb{R}^{8 \times 768}$, $B \in \mathbb{R}^{768 \times 8}$. Only ~700K params are trainable. Fine-tune on a Python code dataset for 10 minutes. Save the adapter (~2MB vs 500MB for the full model). *This is how production LLM fine-tuning works.*

3 Comparison & Experiments

Put both models side-by-side on the same prompts. nanoGPT: ~10M params, trained from scratch on WikiText-103. GPT-2+LoRA: 40× larger, leverages 40B tokens of knowledge, redirected to Python in 10 minutes. Experiments: vary LoRA rank ($r \in \{2, 8, 16\}$), extend nanoGPT training to 5000 steps, swap LoRA target modules. See scaling laws made concrete.

Key Concepts Practised

- Decoder-only transformer (GPT architecture)
- Pre-norm vs post-norm LayerNorm
- Cosine learning rate schedule with warmup
- Language model scaling laws (Kaplan, Chinchilla)
- LoRA: low-rank weight decomposition $\Delta W = BA$
- Parameter-efficient fine-tuning (PEFT) with HuggingFace

Experiments to Try

- In Session 6A, increase MAX_STEPS from 1500 to 3000 — how much does val perplexity improve?
- LoRA rank sweep: train $r \in \{2, 8, 16\}$ — plot quality vs trainable parameter count
- Add "c_proj" to LoRA target_modules — does generation improve? How many extra params?
- Try a different LoRA domain: load your own text corpus and fine-tune GPT-2 on it

Duration: ~45 min each

GPU: Required (Colab T4)

Libraries: peft, transformers, huggingface_hub

Notebooks: [Session_6A_nanoGPT_From_Scratch.ipynb](#) | [Session_6B_GPT2_LoRA_Finetuning.ipynb](#)